Introduction to Machine Learning

Unit 1 notes · Computer Science, Year 2

1. What Is Machine Learning

Machine learning is a branch of artificial intelligence where computers learn patterns from data instead of following rules written by hand. In normal programming, a person writes the rules and the computer applies them to data to get answers. In machine learning, we give the computer data together with the answers, and it works out the rules by itself.

The result of this learning is called a model. A model is a function that takes an input, such as the size and location of a house, and produces an output, such as its predicted price. The more good examples a model sees, the better its predictions usually become.

Machine learning is used when the rules are too complex to write by hand. Examples include filtering spam emails, recommending songs, recognising faces in photos, detecting fraud in M-Pesa transactions, and predicting crop yields from weather data.

There are three main types of machine learning. In supervised learning the data comes with correct answers, called labels. In unsupervised learning there are no labels, and the model looks for hidden groups or patterns, such as grouping customers with similar shopping habits. In reinforcement learning an agent learns by trial and error, receiving rewards for good actions, the way a game-playing program improves by playing many games.

2. Supervised Learning: Classification and Regression

Supervised learning is the most common type of machine learning. Each example in the training data has features, which are the inputs, and a label, which is the correct output. For example, an email's features might be the words it contains, and its label is either spam or not spam.

There are two kinds of supervised learning problems. Classification predicts a category, such as spam or not spam, pass or fail, or which disease a patient has. Regression predicts a number, such as a house price, tomorrow's temperature, or a student's exam score.

Common supervised learning algorithms include linear regression for predicting numbers, logistic regression for yes or no decisions, decision trees that ask a series of questions to reach an answer, and k-nearest neighbours, which predicts by looking at the most similar examples it has already seen.

A common mistake is to think that logistic regression is used for regression. Despite its name, logistic regression is a classification algorithm: it outputs the probability that an example belongs to a class, and that probability is turned into a yes or no answer.

3. Training, Testing and Overfitting

To know whether a model is any good, we must test it on data it has never seen. So before training, the data is split into two parts: a training set, usually about 80 percent of the data,

which the model learns from, and a test set, the remaining 20 percent, which is kept aside to measure how well the model works on new examples.

Training means the algorithm adjusts the model's internal settings, called parameters, to reduce its mistakes on the training set. Testing means using the finished model to predict the test set and comparing its predictions with the true labels. For classification, a simple measure is accuracy: the share of predictions that are correct.

Overfitting happens when a model memorises the training data, including its noise, instead of learning the general pattern. An overfitted model scores very well on the training set but badly on the test set, like a student who memorises past paper answers but cannot answer a new question. Underfitting is the opposite: the model is too simple to learn the pattern and does badly on both sets. Using more data, simpler models, and checking test scores helps avoid overfitting.

The short Python example below uses the scikit-learn library to split data, train a logistic regression model, and measure its accuracy on the test set.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# X = features (inputs), y = labels (correct answers)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression()
model.fit(X_train, y_train)          # training
predictions = model.predict(X_test)  # testing on unseen data
print("Accuracy:", accuracy_score(y_test, predictions))
```

If the training accuracy is much higher than the test accuracy, the model is probably overfitting.